



Resource leveling with subcontracting: algorithms and complexity

Nicholas G. Hall¹ · Marc E. Posner²

Received: 23 August 2024 / Accepted: 12 October 2025 / Published online: 1 December 2025
© The Author(s) 2025

Abstract

A central problem in project management is the optimal use of limited resources. We consider the resource leveling problem, which minimizes the maximum resource load over the life of a project. This problem is motivated by the need to limit expensive resource acquisition. For several variants of this problem, we either provide an efficient optimal algorithm or show that the problem is intractable. We also consider a generalization of the resource leveling problem where work can be subcontracted. The subcontracting cost is defined either by task or by time of subcontracting. Our results show how the solvability of resource leveling problems depends on whether task times are unit or arbitrary, whether resource requirements are unit or binary, whether the project network is a chain, a tree, or an opposing forest, whether subcontracting costs for jobs are unit or binary, and whether the availability of subcontracting at a time period is constrained.

Keywords Project management · Minimizing maximum resource usage · Outsourcing · Algorithms

Mathematics Subject Classification Project management · Production/scheduling

1 Introduction

A project is a “temporary endeavor undertaken to create a unique product or service” (Project Management Institute 2013). The impact of project management is estimated at 30% of global economic activity (Hu et al. 2015), with an annual value of about \$27 trillion. Project management applications range from traditional ones including construction and engineering to diverse modern ones such as information technology, research and development, new product and service development, and corporate change management. This work considers the *resource leveling problem* (RLP) in project management. The context of the RLP includes two other well studied project management problems that we now briefly describe.

First, the resource-constrained project scheduling problem (RCPSP) takes project resources as given and *minimizes the project makespan*. A survey of research on this problem can be found in Demeulemeester and Herroelen (2002).

The RCPSP is highly intractable (Blazewicz et al. 1983). Because of its practical importance, commercial software has been developed for this problem, although its effectiveness is limited. A study by Kolisch (1999) documents heuristic errors ranging from 4% to 10% in finding the shortest project schedule by the best known project planning software packages. Browning and Yassine (2010) compare several heuristic scheduling rules using five objective measures and over 12,000 test problems, from which they conclude that widely used rules often perform poorly.

Second, the Resource Availability Cost Problem (RACP) (Demeulemeester 1995), also named the Resource Investment Problem (Drexl and Kimms 2001), allows varying resource costs over time. The objective is to *minimize the total cost associated with the use of the resources*. Möhring (1984) identifies sets of feasible partial orders that extend the given precedence relations, and uses interval graph recognition to develop algorithms. He reports computational experience with 16 tasks and four resource types. Demeulemeester (1995) uses maximum and minimum bounding strategies to identify non-dominant resource usage vectors, and solves problems with 20 tasks and six resource types.

By contrast, the RLP takes the project makespan as fixed and minimizes the maximum resource load employed over the life of a project. The objective is to *minimize the max-*

✉ Nicholas G. Hall
hall.33@osu.edu

¹ Fisher College of Business, The Ohio State University, Columbus, USA

² Department of Integrated Systems Engineering, The Ohio State University, Columbus, USA

imum amount of resource required. This objective is partly motivated by minimizing the cost of resources that are dedicated to the project throughout its life. The use of dedicated resources is common in projects that are organized with a high level of autonomy, known as a project team (Hobbs and Hobbs 1993), to motivate project workers and encourage creativity. An additional motivation is to ensure that expensive short-term procedures such as the allocation of additional resources to the project, or the outsourcing of work to third-party providers, can be avoided.

Heuristic studies of the RLP are initiated by Burgess and Killebrew (1962). Kyriklidis and Dounias (2016) review the recent literature of solution approaches for the RLP. These approaches can be classified into: classical optimization, including enumeration (Younis and Saad 1996), dynamic programming (Elwany et al. 1998); fast and simple heuristic rules (Neumann and Zimmermann 1999, 2000); and computationally intensive search methods such as neural networks (Kartam and Tongthong 1998) and Petri nets (Raja and Kumanan 2007). Exact solutions for the RLP remain difficult to find for projects larger than 50 tasks (Rieck and Zimmermann 2014). The need to solve large problems accurately in practice motivates our work.

Our work considers the efficient solvability of various cases of the RLP, including several that allow for the subcontracting of tasks. Our first contribution is a mapping of the boundary between efficiently solvable and formally intractable cases of the RLP. Besides enabling the solution of some cases, this mapping also justifies heuristic solution procedures for other cases. Further, in considering subcontracting problems under two different definitions, our work shows to what extent subcontracting can alleviate bottlenecks in RLPs. Once again, we identify variants of the problem that are efficiently solvable.

The remainder of this paper is organized as follows: Section 2 provides our notation and describes the objective and related assumptions. In Section 3, we establish the efficient solvability or intractability of several variants of the RLP. A generalization of the RLP where work can be subcontracted is studied in Section 4. Finally, Section 5 contains a summary of the results and some suggestions for future research.

2 Preliminaries

There are $N = \{1, \dots, n\}$ tasks to be scheduled. We first allow only nonpreemptive, or uninterrupted, processing of the tasks. Each task $j \in N$ has a processing time p_j , and a constant resource requirement r_j throughout its duration. In the RLP, the project completion time T is modeled as a hard constraint. We let Ω denote the set of feasible problem instances, and $I_n \in \Omega$ be an instance where there are n tasks.

Since rational data can be rescaled, we assume that all data are integer and nonnegative.

Using the standard 3-field notation $\alpha|\beta|\gamma$ (Graham et al. 1979), we define the following notation:

Under α :

PS : project scheduling problem

P : arbitrary number of parallel machines

Pm : a constant number m of parallel machines.

Under β :

r_j : resource requirement throughout task j

$p_j = 1$: each task has unit processing time

$pmtn$: task preemption is allowed.

Under γ :

$R_{\max}$: minimize the maximum resource used in any period.

Because RLPs have precedence constraints, we include them in our discussion. The types of precedence constraint that are considered under β are:

chains: precedence constraints are a set of chains

intree: precedence constraints are an intree

tree: precedence constraints are a tree

opffor: precedence constraints form an opposing forest

{empty}: arbitrary directed acyclic graph.

Since the problem we study is motivated within the context of project management, the default assumption of an arbitrary precedence graph is not stated. Similarly, we assume for all problems that $C_{\max} \leq T$ when $R_{\max}$ appears in the γ field. While the analysis of intrees and outtrees can be considerably different, for the problems we consider, if time is reversed, then our intree analysis can be used to solve a corresponding outtree problem. Since a project management network is conventionally defined to include a sink node that represents the completion of the project, we only consider intrees. An opposing forest is a set of trees that contains at least one intree and one outtree.

3 Solvability of RLP

In this section, we provide complexity results for problems that are close to the boundaries of the various complexity classes. We first show that all RLPs where $r_j \in \{0, 1\}$ are intractable. The reduction is from the following problem which is known to be unary NP -complete (Garey and Johnson 1979).

Exact Cover by 3-Sets: Given a set X where $|X| = 3u$ for some positive integer u and a collection J of 3-element subsets of X , does there exist a subcollection $J' \subseteq J$ such that every element of X occurs in exactly one member of J' ?

Theorem 1 *The recognition version of problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$ is unary NP-complete.*

Proof Given an arbitrary instance of Exact Cover by 3-Sets, we construct an instance of problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$. Let the makespan of the RLP be $T = (|J| - u + 1)(3u + 2Ku + 1)$, where $K > 4|J|/u$ and integer. Also, let the objective function threshold, which is the maximum resource level, be $L = u$.

We construct four types of chains in pseudopolynomial time. To simplify their description, we index the tasks within each chain in precedence order.

Type 1: There are $u - 1$ chains of length T , where tasks $3u + 1$ and $3u + Ku + 2, \dots, 3u + 2Ku + 1$ have $r_j = 0$, and all remaining tasks have $r_j = 1$.

Type 2: There is one chain of length T , where tasks $3u + 2, \dots, 3u + Ku + 1$ have $r_j = 1$, and the remaining tasks have $r_j = 0$.

Type 3: There is one chain of length $(3u + 2Ku + 1) + (|J| - u)(3u + Ku - 3)$, where tasks $1, \dots, 3u + 2Ku + 1$ have $r_j = 0$, and the remaining $(|J| - u)(3u + Ku - 3)$ tasks have $r_j = 1$.

Type 4: There are $|J|$ chains of length $3u + 2Ku + 1$. Each consists of a chain of length $3u$, followed by 1 task where $r_j = 1$, followed by Ku tasks where $r_j = 0$, followed by Ku tasks where $r_j = 1$. The resource requirements for tasks $1, \dots, 3u$ in each chain are constructed from a different subset in the collection J . Specifically, $r_j = 1$ if the j th element of X is in the corresponding subset, and $r_j = 0$ otherwise.

As a result, the number of tasks is

$$\begin{aligned} & (u - 1)T + T + (3u + 2Ku + 1) + (|J| - u)(3u + Ku - 3) \\ & + |J|(3u + 2Ku + 1) \\ & = (3u + 2Ku + 1)[(u + 2)|J| - u^2 + 1] \\ & - (|J| - u)(Ku + 4). \end{aligned}$$

Because the Type 1 and Type 2 chains have length T , each is scheduled throughout the interval $[0, T]$. From the definition of L , the remaining resource capacity that is available for the Type 3 and Type 4 chains is 1, for periods $1, \dots, 3u$, u for period $3u + 1$, 0 for periods $3u + 2, \dots, 3u + Ku + 1$, u for periods $3u + Ku + 2, \dots, 3u + 2Ku + 1$, and 1 for the remaining periods $3u + 2Ku + 2, \dots, T$. Observe that the total units of available resource once the Type 1 and Type 2

chains are assigned is $3u + u + Ku^2 + [T - (3u + 2Ku + 1)] = (|J| - u)(3u + Ku - 3) + |J|(Ku + 4)$, which is exactly the total resource requirement of the Type 3 and Type 4 chains. Consequently, a feasible schedule must use all of the available resources and contain no idle time. Without loss of generality, we assume that when a task with $r_j = 0$ has no unprocessed predecessors, it is processed as early as possible.

To complete the proof, we show that the constructed instance of problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$ has a feasible solution with a makespan of T and an objective value of $R_{\max} \leq L$ if and only if there exists an exact cover of X by 3-sets.

($\Rightarrow$) Assume the existence of an exact cover by 3-sets. Let $S_1, \dots, S_u$ denote the 3-element subsets of J that form an exact cover of X . Schedule all the Type 4 chains corresponding to subsets $S_1, \dots, S_u$ to start processing at time 0 and complete at time $3u + 2Ku + 1$. Observe that all the available resources are used in the interval $[0, 3u + 2Ku + 1]$. Now, schedule the remaining $(|J| - u)$ chains of Type 4 consecutively in any sequence without idle time, starting at time $3u + 2Ku + 1$ and finishing at time $(3u + 2Ku + 1) + (|J| - u)(3u + 2Ku + 1) = T$. Observe that this does not require more than the remaining available capacity of 1 at any time in the interval $[3u + 2Ku + 1, T]$. However, one unit of idle time occurs at exactly $(|J| - u)(3u + Ku - 3)$ times within that interval. Now, schedule the first $(3u + 2Ku + 1)$ tasks of the Type 3 chain consecutively from time 0. The remaining tasks of the Type 3 chain become available one after another, and all have $r_j = 1$. Therefore, one of the remaining $(|J| - u)(3u + Ku - 3)$ tasks in the Type 3 chain is scheduled whenever there is an idle time left by the Type 4 chains. Thus, there exists a feasible schedule for problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$ with makespan T and $R_{\max} \leq L$.

($\Leftarrow$) Assume there exists a feasible schedule for problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$ with a makespan of T and an objective value $R_{\max} \leq L$. Because interval $[0, 3u]$ has a total resource availability of $3u$, and each Type 4 chain requires three units of resource in the first $3u$ periods, it is not possible that $(u + 1)$ or more Type 4 chains complete their first $3u$ tasks by time $3u$.

Suppose that $(u - 1)$ or fewer Type 4 chains complete their first $3u$ tasks by time $3u$. Observe that any Type 4 chain that does not complete its first $3u$ tasks by time $3u$ cannot complete its $(3u + 1)$ st task until at least time $3u + Ku + 2$. Therefore, that chain cannot schedule any later task with $r_j = 1$ until at least time $3u + 2Ku + 2$. As a result, the Ku tasks $\{3u + Ku + 2, \dots, 3u + 2Ku + 1\}$ with $r_j = 1$ in the Type 4 chains that complete their first $3u$ tasks by time $3u$ can use at most $Ku(u - 1)$ of the available Ku^2 resources in the interval $[3u + Ku + 1, 3u + 2Ku + 1]$. The remaining Ku units of resource must be used by the first $3u + 1$ tasks of any Type 4 chains. For each of these chains, at most four of their

tasks have $r_j = 1$. Hence, the use of all available resources requires that $4|J| \geq Ku$, which contradicts the definition of K .

Consequently, exactly u Type 4 chains must complete their first $3u$ tasks by time $3u$. The total resource requirement of the first $3u$ tasks of these u chains is $3u$, and each period of $1, \dots, 3u$ has a resource availability of 1. Then, the construction of the r_j values in the Type 4 chains implies the existence of an exact cover of X by 3-sets. $\square$

As a result of Theorem 1, we restrict our attention to RLPs where $r_j = 1$. Several complexity results for the RLP can be derived from existing results for Classical Resource Scheduling (CRS) problems. These problems have a classical objective function, such as $C_{\max}$, i.e. makespan. When $r_j = 1$, results from CRS problems can be used to establish the complexity of corresponding RLPs.

Theorem 2 *The CRS problem $P \mid \beta \mid C_{\max}$ is solvable in polynomial time if and only if $PS \mid \beta, r_j = 1 \mid R_{\max}$ problem is polynomially solvable.*

Proof ($\Rightarrow$) Suppose problem $P \mid \beta \mid C_{\max}$ is polynomially solvable. Find the minimum number of machines, m , required to satisfy $C_{\max} \leq T$ in problem $Pm \mid \beta \mid C_{\max}$. This value of m can be found using binary search by solving $O(\log n)$ CRS problems. Since the CRS problem instance is polynomially solvable, m can be determined in polynomial time. Because $r_j = 1$ for $j \in N$, m is also the minimum value of the maximum resource requirement in $PS \mid \beta, r_j = 1 \mid R_{\max}$.

($\Leftarrow$) Suppose problem $PS \mid \beta, r_j = 1 \mid R_{\max}$ is polynomially solvable. Set the number of machines in the CRS problem to the optimal value of the RLP. Using binary search on the value of T , find the smallest feasible value of T in the CRS problem. This can be accomplished by solving $O(\log \sum p_i)$ CRS problems. Since the RLP is polynomially solvable, the value for T can be found in polynomial time. The value of T found is the minimum makespan for $P \mid \beta \mid C_{\max}$. $\square$

Corollary 1 *If the recognition version of CRS problem $P \mid \beta \mid C_{\max}$ is binary (respectively, unary) NP-complete, then the recognition version of $PS \mid \beta, r_j = 1 \mid R_{\max}$ is also binary (resp., unary) NP-complete.*

Proof Follows from the ($\Leftarrow$) part of Theorem 2. $\square$

We use the following observation about the complexity relationships between certain preemptive and non-preemptive RLPs.

Remark 1 The recognition version of an RLP where preemption is allowed and p_j is arbitrary is at least as hard as the corresponding RLP with no preemptions and $p_j = 1$.

Proof A special case of the preemptive problem is a problem where $p_j = 1$ for $j \in N$. Since all data is integer, there exists an optimal solution to the special case where no task is preempted. $\square$

Some classical scheduling results provide complexity analysis of RLPs. Du et al. (1991) show that the recognition version of problem $P2 \mid chain \mid C_{\max}$ is unary NP-complete, which from Corollary 1 implies a similar result for problem $PS \mid chain, r_j = 1 \mid R_{\max}$. Also, Garey et al. (1983) show that the recognition version of $P \mid oppfor, p_j = 1 \mid C_{\max}$ is unary NP-complete. Then, Corollary 1 implies that the recognition version of problem $PS \mid oppfor, p_j = 1, r_j = 1 \mid R_{\max}$ is unary NP-complete. Further, Remark 1 establishes that the recognition version of $PS \mid oppfor, pmtn, r_j = 1 \mid R_{\max}$ is also unary NP-complete.

Some classical results that provide polynomial-time algorithms can also be applied to RLPs. Hu (1961) directly shows that $PS \mid intree, p_j = 1, r_j = 1 \mid R_{\max}$ is solvable in $O(n)$ time. Further, the results of Muntz and Coffman (1970) show that $P \mid intree, p_j = 1 \mid C_{\max}$ is solvable in $O(n)$ time. Then, the ($\Rightarrow$) part of Theorem 2 shows that $PS \mid intree, pmtn, r_j = 1 \mid R_{\max}$ is solvable in $O(n \log n)$ time. Finally, McNaughton (1959) shows that problem $P \mid pmtn \mid C_{\max}$ is solved in $O(n)$ time by a “wrap around” procedure. Problem $PS \mid chain, p_j = 1, r_j = 1 \mid R_{\max}$ can be transformed into this problem, where each chain h of size n_h is represented by a job $j = h$ with processing time $p_j = n_h$. The minimum resource level is $R_{\max} = \lceil n/T \rceil$.

Table 1 presents results for RLPs that are near the complexity boundary. The first column shows the type of precedence graph. The second column shows whether preemption of tasks is allowed. The third column specifies whether processing time is arbitrary or $p_j = 1$ for all tasks. Since the data are integral, when $p_j = 1$ in the third column, preemption is not applicable in the second column and is hence denoted by “n.a.”. Also, in the third column, some results apply to both cases arbitrary p_j and $p_j = 1$ and are therefore denoted “Any”. In the fourth column, either some tasks may have $r_j = 0$ and others may have $r_j = 1$, or all tasks have $r_j = 1$. Included in the appropriate cell of the final column is the complexity result and a reference to where a proof of that result can be found.

4 Subcontracting

Subcontracting is a common practice in projects. For the last 30 years, organizations have been outsourcing their activities with increasing frequency (Pralhad and Hamel 1990; Quinn and Hilmer 1994; Tully 1994). The outsourced activities may range from routine business processes including

Table 1 Complexity of boundary problems

Precedence	Preemption	$p_j = 1$	r_j	Complexity result	
Chains	N.a.	Yes	$\in \{0, 1\}$	Unary <i>NPC</i>	Thm. 1
	No	No	1	Unary <i>NPC</i>	Du et al. (1991), Cor. 1
Opposing	N.a.	Yes	1	Unary <i>NPC</i>	Garey et al. (1983), Rem. 1
Forest	Yes	Any	1	Unary <i>NPC</i>	Garey et al. (1983), Thm. 2
Intree	N.a.	Yes	1	$O(n)$	Hu (1961)
	Yes	Any	1	$O(n \log n)$	Muntz and Coffman (1970), Thm. 2

payroll processing and information technology (Crail 2024), to the manufacturing of components (Sturgeon 2002), to special creative activities related to new product development (Pisano and Verganti 2008). When there is variability in the demand, outsourcing reduces capital requirements. Furthermore, in many projects, it is not economically viable for a project company to maintain sufficient expertise that is only needed occasionally in-house. In these cases, specialized tasks are usually performed by outside subcontractors with the relevant expertise. For descriptive convenience, we say that if a task is *processed* in an interval, then it is either handled by the in-house machine resource or is subcontracted.

The possibility of subcontracting defines two new problem types within the class of RLPs, based on how subcontracting costs are defined. For the first type, we describe an Algorithm SubTreeC. For the second type, we describe a Procedure LSubChainS. In both cases, we provide related intractability results. We now formally state one of the problems.

Subcontracting Problem A: *Given a set of tasks with a partial precedence order, a global deadline T , a cost $c_j \geq 0$ for subcontracting task $j \in N$, and an integer subcontracting budget $B \geq 0$, find a schedule that minimizes the maximum integer period resource usage $R_{\max}$, where not more than B is spent on subcontracting and where $C_{\max} \leq T$.*

We denote this subcontracting problem by including c_j in the second field.

Remark 2 If $B = 0$ and $c_j > 0$ for $j \in N$, then Subcontracting Problem A reduces to the original RLP. Hence, all intractability results derived in Section 3 hold for the subcontracting problem.

In view of Remark 2, we focus on RLPs that are polynomially solvable and discuss the solvability of their counterparts that include subcontracting.

Theorem 3 *The recognition version of problem $PS|chains, p_j = 1, r_j = 1, c_j \in \{0, 1\} | R_{\max}$ is unary NP-complete.*

Proof By reduction from problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$, for which the recognition version is shown to be unary NP-complete in Theorem 1.

Given an arbitrary instance of problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$, we construct an instance of problem

$PS|chains, p_j = 1, r_j = 1, c_j \in \{0, 1\} | R_{\max}$ with the following data. The tasks, precedence graph, makespan, and threshold value of $R_{\max}$ are unchanged. For any task j , set $c_j = 0$ when $r_j = 0$, and set $c_j = 1$ when $r_j = 1$. Further, let $B = 0$.

Without loss of generality, all tasks where $c_j = 0$ are subcontracted. However, the subcontracting of any task j where $c_j = 1$ is not feasible. Hence, those tasks must be processed by the machines. Thus, the tasks that are not subcontracted are exactly those that have $r_j = 1$ in the RLP. Therefore, only those tasks must be scheduled to meet the threshold for the number of tasks processed concurrently.

It follows that there exists a “yes” answer to the recognition question for the constructed instance of problem $PS|chains, p_j = 1, r_j = 1, c_j \in \{0, 1\} | R_{\max}$ if and only if there exists a “yes” answer to the recognition question for problem $PS|chains, p_j = 1, r_j \in \{0, 1\} | R_{\max}$. $\square$

Theorem 3 and Remark 2 suggest that many subcontracting problems are NP-complete. This motivates the examination of special cases. The first problem we consider is $PS|\pi, p_j = 1, r_j = 1, c_j = 1 | R_{\max}$, for $\pi \in \{intree, outtree\}$, where all tasks have unit subcontracting costs. As mentioned in Section 2, if time is reversed, then our analysis of intree problems and can be used for outtree problems. As a result, we only consider the intree problem.

We extend the algorithm of Hu (1961) for the classical scheduling problem $P|intree, p_j = 1 | C_{\max}$. Similar to Hu (1961), the *level* of task $j \in N$ is $d(j)$, which is the length, i.e. the number of arcs, of the unique path from that task to the terminal node of the intree. Also, let $\bar{d} = \max_{j \in N} \{d(j)\}$. As in Hu, in a given partial schedule, we define the set of *available tasks* as those tasks which have not started processing, and for which all predecessors are complete. These tasks correspond to the leaf nodes of the current partial precedence graph.

Remark 3 (Hu 1961) For problem $PS|intree, p_j = 1, r_j = 1, c_j = 1 | R_{\max}$, the number of available tasks is nonincreasing over time in every feasible schedule.

Let $\rho_t(I) = |\{j \in N \mid d(j) = T - t\}|$ for problem instance I and $t \in \mathcal{T} = \{1, \dots, T\}$. For a schedule to be feasible, the set of tasks $\{j \in N \mid d(j) = T - t\}$ must start processing by time t . When there is no ambiguity, the I is removed. Further, for a given number of machines $L \in \{0, \dots, n\}$, let $\tau(L) = \max\{t \in \mathcal{T} \mid \rho_t(I) > L\}$. Once period $\tau(L) + 1$ is reached, no further subcontracting is needed, since the number of machines is at least as large as the number of available tasks. This suggests the following result.

Lemma 1 *Given L machines, at least $\max\{0, \sum_{t=1}^{\tau(L)} \rho_t - L\tau(L)\}$ tasks require subcontracting in a feasible solution to problem $PS \mid \text{intree}, p_j = 1, r_j = 1, c_j = 1 \mid R_{\max}$.*

Proof From Remark 3, no subcontracting is needed for the intree for periods $\tau(L) + 1, \dots, T$. To obtain a feasible solution, the total processing needed for periods $1, \dots, \tau(L)$ is $\sum_{t=1}^{\tau(L)} \rho_t$. A maximum of $L\tau(L)$ tasks can be processed by L machines during this period. Hence, at least $\sum_{t=1}^{\tau(L)} \rho_t - L\tau(L)$ tasks require subcontracting. $\square$

The next procedure, Algorithm SubTreeC, finds an optimal solution to Problem $PS \mid \text{intree}, p_j = 1, r_j = 1, c_j = 1 \mid R_{\max}$ in strongly polynomial time (see Grötschel et al. 1988). Lemma 1 shows that $L\tau(L)$ tasks can be processed during periods $1, \dots, \tau(L)$. The intuition behind the algorithm is to delay subcontracting as long as possible, i.e., only subcontract when failure to do so would result in a terminal task violating the deadline. The period is represented by t . When $t > T$, the project completion time is exceeded and no further processing is possible.

Algorithm SubTreeC

0. Input n, T, B , and the intree precedence graph.
1. Find the level $d(j)$ of each task $j \in N$. Set $\bar{d} = \max_{j \in N} \{d(j)\}$.
Let W = set of tasks without predecessors, and $w = |W|$.
Calculate ρ_t , for $t = 1, \dots, T$, and $\tau(l)$, for $l = 0, \dots, w$.
Find $L = \min\{l \in \{0, \dots, w\} \mid \sum_{t=1}^{\tau(l)} \rho_t - l\tau(l) \leq B\}$.
Set $t = 1$.
2. Use the machines to process any $\min\{L, w\}$ tasks with largest $d(\cdot)$ values.
If all tasks with level $\bar{d}$ are now processed, then set $\bar{d} = \bar{d} - 1$.
If $w \leq L$ or if $\bar{d} \leq T - t$, then go to Step 3.
Otherwise, subcontract the jobs of the set $A = \{j \in W \mid d(j) = \bar{d}, \text{ task } j \text{ is unscheduled}\}$. Set $\bar{d} = \bar{d} - 1$.
3. Remove the processed and subcontracted tasks from the tree and update W and w .
Set $t = t + 1$.
If $t \leq T$, then go to Step 2.
Otherwise, output the value of $R_{\max} = L$ and the associated schedule, and stop.

Observe that the selection of L in Step 1 is feasible. In what follows, it is shown that L is an optimal selection. Also note that in Step 2, the tasks with the largest $d(\cdot)$ values are selected for processing, which are those tasks $j \in N$ where $d(j) = \bar{d}$. However, when this type of task is not available, it is necessary to process tasks where $d(j) < \bar{d}$.

To show that Algorithm SubTreeC finds a feasible schedule, we need a preliminary result that establishes a relationship between instances of a classical intree problem.

Lemma 2 *Consider two instances, I_1 and I_2 of problem $PS \mid \text{intree}, p_j = 1, r_j = 1, c_j = 1 \mid R_{\max}$, where both instances have the same number of tasks and value of T . Suppose that $\sum_{i=1}^j \rho_{I_1}(i) \leq \sum_{i=1}^j \rho_{I_2}(i)$ for $j = 1, \dots, T$. Then, an optimal solution to I_1 does not require more resources than an optimal solution to I_2 .*

Proof Hu (1961) shows that given L machines,

$$L \geq \frac{\sum_{i=1}^j \rho_{\bar{d}-i+1}}{T - \bar{d} + j}, \quad j = 1, \dots, \bar{d} - \tau(L) \quad (1)$$

is a necessary and sufficient condition for the existence of a feasible solution without subcontracting. If $\sum_{i=1}^j \rho_i(I_1) \leq \sum_{i=1}^j \rho_i(I_2)$ for all j , then whenever (1) is satisfied for I_2 , (1) is also satisfied for I_1 . $\square$

Intuitively, Lemma 2 states that an intree is easier to process if the cumulative number of tasks at each level from the top of the tree is no larger than in another intree.

Remark 4 The number of machines required to solve an instance of $PS \mid \text{intree}, p_j = 1, r_j = 1, c_j = 1 \mid R_{\max}$, depends only on the number of tasks at each level and not on the precedence structure between tasks of adjacent levels.

We now establish that Algorithm SubTreeC finds an optimal schedule.

Theorem 4 *Algorithm SubTreeC finds an optimal schedule for problem $PS \mid \text{intree}, p_j = 1, r_j = 1, c_j = 1 \mid R_{\max}$ in $O(n)$ time.*

Proof Lemma 1 specifies a lower bound on the total amount of subcontracting needed. This requirement is satisfied in Step 1 of Algorithm SubTreeC.

Each iteration of Algorithm SubTreeC increases t by 1. As a result, the algorithm stops after T iterations. Step 2 ensures that no subcontracting occurs until $t = T - \bar{d} + 1$. From this time forward, at least one task level is completed each period, using subcontracting when necessary. Consequently, after T iterations, all tasks at every level are processed. As a result, Algorithm SubTreeC finds a solution that satisfies the completion time constraint.

Once $w \leq L$, Step 2 of Algorithm SubTreeC creates no additional subcontracting in the current or any subsequent

periods. From Remark 3 and the fact that Step 2 always processes the tasks with the largest d values first, the condition that $w \leq L$ occurs no later than time $t = \tau(L) + 1$. Hence, subcontracting can occur only in periods $1, \dots, \tau(L)$.

The selection of tasks to be processed follows Lemma 2 and Remark 4, by always selecting those available tasks at the highest level. As a result, the selection of tasks maximizes the amount of processing by the machines.

From Step 2, if subcontracting occurs first at time t' , then this is the first time that $\bar{d} = T - t' + 1$ and $\rho_{t'} > L$. Remark 3 shows that L tasks are processed in each period $1, \dots, t'$. Step 2 continues to process L tasks each period until the number of available tasks $w < L$. Since this occurs at a period no later than $\tau(L)$, Lemma 1 establishes the optimality of the schedule found by Algorithm SubTreeC.

Step 1 calculates and stores all $\tau(l)$ and $\sum_{t=1}^{\tau(l)} \rho_t$ values for $l = 1, \dots, T$ in $O(n)$ time. Using binary search, the value of L can be found in $O(\log n)$ time. Steps 2 and 3 are repeated $T = O(n)$ times. Scheduling or subcontracting each task and updating the list of available tasks require $O(n)$ time. If $T \geq n$, the problem is trivial because $L = 0$ when $B \geq n$ and 1 otherwise. If $T < n$, then the overall computation time of Algorithm SubTreeC is $O(n)$. $\square$

When the subcontracting costs are associated with a time period rather than with a task, the subcontracting problem is slightly easier. Let s_t be the cost of subcontracting a task in period $t \in \mathcal{T}$. In the 3-field notation, we denote this problem with an s_t in the second field of the subcontracting problem. We state the problem formally.

Subcontracting Problem B: Given a set of tasks with a partial precedence order, a global deadline T , a cost s_t for subcontracting any task in period $t \in \mathcal{T}$, and an integer subcontracting budget B , find a schedule that minimizes the maximum period resource usage $R_{\max}$, where not more than B is spent on subcontracting and where $C_{\max} \leq T$.

Given a graph with n tasks that form q chains, let $Q = \{1, \dots, q\}$, and let chain h have n_h tasks, for $h \in Q$. Thus, $\sum_{h \in Q} n_h = n$. We consider a general version of Problem $PS | chains, p_j = 1, r_j = 1, s_t | R_{\max}$, where the allowable amount of subcontracting is restricted by period. Let a_t be the maximum amount of subcontracting that is permitted in period $t \in \mathcal{T}$ and $a = (a_1, \dots, a_T)$. Using the three-field notation, this problem is described as $PS | chains, p_j = 1, r_j = 1, a_t, s_t | R_{\max}$. We show that this scheduling problem is solvable in polynomial time.

If the total available resources $TL \geq n$, then no subcontracting is necessary. As mentioned in Section 3, this problem can be transformed into Problem $P | pmtn | C_{\max}$ and solved in $O(n)$ time (see McNaughton 1959).

Remark 5 To determine a schedule for Problem $PS | chains, p_j = 1, r_j = 1, a_t, s_t | R_{\max}$, it is sufficient to specify the chains that are assigned resources by period.

Remark 5 holds because a given period can assign resources to at most one task in any chain. Once all assignments to chains are complete, the period assignments to a chain provide a time order that uniquely specifies the period–task assignments. As a result of Remark 5, the problem can be formulated as a binary transportation problem with a side constraint. Let $x_{th} = 1$ if time period $t \in \mathcal{T}$ assigns one machine or subcontracting resource to chain $h \in Q$, and $x_{th} = 0$ otherwise.

Problem $PS | chains, p_j = 1, r_j = 1, a_t, s_t | R_{\max}$ can be formulated as

$$\begin{aligned} \min L \quad & (P_{SC}) \\ \text{sub. to} \quad & \sum_{t \in \mathcal{T}} x_{th} = n_h, \quad h \in Q \end{aligned} \quad (2)$$

$$\sum_{j \in Q} x_{tj} \leq a_t + L, \quad t = 1, \dots, T \quad (3)$$

$$\sum_{t \in \mathcal{T}} s_t \max\left\{\sum_{h \in Q} x_{th} - L, 0\right\} \leq B \quad (4)$$

$$x_{th} \in \{0, 1\}, \quad t \in \mathcal{T}; h \in Q.$$

Equation (2) ensures that each task on chain $h \in Q$ is assigned a resource. Inequality (3) provides the restriction on available resources for period $t \in \mathcal{T}$. Finally, Inequality (4) represents the budget constraint.

Our approach for solving P_{SC} is to develop a procedure for finding a minimum cost solution for a given value of L . Then, this procedure is applied using binary search on L , where $L \in \{0, 1, \dots, q\}$. The following procedure finds a schedule for P_{SC} that uses a minimal budget, where L is fixed.

Procedure LSubChainS

0. Input T, L, a, s, q , and n_h for $h \in \{1, \dots, q\} = Q$.
1. Set $\gamma = \max\{n - LT, 0\}$. If $\gamma = 0$, then go to Step 3.
 - Sort the chains such that $n_1 \geq \dots \geq n_q$. Set $\bar{n}_h = n_h$ for $h \in Q$, and $\bar{q} = q$.
 - Sort the period indices such that $s_{i_1} \leq \dots \leq s_{i_T}$.
 - Set $k = 1$.
2. For period i_k , let $u = \min\{L + a_{i_k}, L + \gamma, \bar{q}\}$ and assign a total of L resources and $u - L$ subcontracting tasks to chains $1, \dots, u$ in any order.
 - Set $\gamma = \gamma - (u - L)$.
 - If $\gamma = 0$, then go to Step 3.
 - Set $\bar{n}_h = \bar{n}_h - 1$ for $h = 1, \dots, u$.
 - If $\bar{n}_h = 0$, then set $\bar{q} = \bar{q} - 1$, for $h = 1, \dots, u$.
 - Set $k = k + 1$. If $k > T$, then output assigned subcontracting and resources, and stop.
 - Resort the chain indices such that $\bar{n}_1 \geq \bar{n}_2 \geq \dots \geq \bar{n}_{\bar{q}}$, and go to Step 2.
3. Use the “wrap-around” procedure of McNaughton (1959) to assign resources to the remaining chains that require

resources.

For each chain, assign the period resources to the individual tasks. Output assigned subcontracting and resources, and stop.

When $n > LT$, LT tasks are satisfied by the resources without subcontracting. In Step 1 of Procedure LSubChainS, the total amount of subcontracting required, γ , is calculated. Step 2 ensures that exactly γ tasks are subcontracted by maximizing the subcontracting from each period selected. When $t > T$, then all periods have been assigned processing or subcontracting. In Step 2, the assignments by time period are identified only by the chain to which they belong. The sequencing of the tasks to satisfy chain precedence relations is completed in Step 3.

To establish that Procedure LSubChainS finds a schedule for P_{SC} with minimal cost where L is fixed, additional notation is needed. Let the period indices $i_1, \dots, i_T$ be ordered such that $s_{i_1} \leq \dots \leq s_{i_T}$. Also, let $\bar{q}_k$ denote the value of $\bar{q}$ in iteration k of Procedure LSubChainS, for $k \in \mathcal{T}$. This value is the number of chains that have tasks which have not been scheduled for processing after periods $i_1, \dots, i_{k-1}$ have made their assignments. In Step 1, $\bar{q}_0 = q$. Each time Step 2 is entered, $\bar{q}_k$ is calculated and then k increases by 1. For a given L , calculate $\bar{u} = (\bar{u}_1, \dots, \bar{u}_T)$ as follows:

$$\bar{u}_k = \min\{L + a_{i_k}, L + \gamma, \bar{q}_k\}, \quad k = 1, \dots, T. \quad (5)$$

Both $\bar{q}_k$ and $\bar{u}_k$ depend on the instance I , but the instance is not included in the description when it is clear from context.

Intuitively, $\bar{u}_k$, which has three components, is the maximum number of tasks that can be processed in period i_k . The first component, $L + a_{i_k}$, is the maximum total available resources in period i_k , which consist of machines and subcontracts. The second, $L + \gamma$, is an upper bound on the number of tasks that require processing in period i_k . The third component, $\bar{q}_k$, is the number of chains with tasks that are available for processing in period i_k .

Remark 6 If resources are assigned as in Step 2 of Procedure LSubChainS, then $\bar{q}_0 \geq \bar{q}_1 \geq \dots \geq \bar{q}_T$.

Remark 6 follows from the fact that in Procedure LSubChainS, available tasks are selected by period in the order $i_1, \dots, i_T$ and the number of chains with available tasks does not increase as tasks are scheduled.

We now show that Procedure LSubChainS finds a feasible schedule when one exists. The next result shows the advantage of selecting tasks from the longest chains.

Lemma 3 Given $\bar{a}_i$ for $i \in \{1, \dots, T\}$, if at each stage a maximum number of tasks are assigned to the longest chains, then any order of assignments provides the same assignments to the chains.

Proof Because Procedure LSubChainS has the chains arranged in nonincreasing order of number of tasks without assignments, $\bar{n}_1 \geq \dots \geq \bar{n}_{\bar{q}}$ where $\bar{q}$ is the current set of chains that have unassigned tasks.

Consider the assignment of tasks to two periods i_1 and i_2 , where the task assignments are made for i_2 directly after i_1 . Then, i_1 assigns resources to chains $1, \dots, \bar{u}_1$, and i_2 assigns resources to chains $1, \dots, r$ and $\bar{u}_1 + 1, \bar{u}_1 + 2, \dots, v$, where $r \leq \bar{u}_2$ and $v = \bar{u}_1 + \bar{u}_2 - r$. If $r < \bar{u}_2$, then the i_2 assignments skip chains $r + 1, \dots, \bar{u}_1$. After the i_1 assignments, skipping occurs only when these chains have fewer unassigned tasks than chains $\bar{u}_1 + 1, \bar{u}_1 + 2, \dots, v$. This happens only when $\bar{n}_r > \bar{n}_{r+1} = \dots = \bar{n}_v$. If $\bar{n}_{r+h} > \bar{n}_{r+h+1}$ for some $h \in \{1, \dots, v - r - 1\}$, then chain $r + 1$ would have been assigned.

If i_2 selects chains directly before i_1 , then i_2 selects the chains $1, \dots, \bar{u}_2$ and i_1 selects the chains $1, \dots, \min\{r, \bar{u}_1\}$ and $\bar{u}_2 + 1, \bar{u}_2 + 2, \dots, v$. Thus, the same assignments are made as when period i_1 selects first.

By repeating a series of pairwise interchanges, any order of assignments result in the same selection of chains. $\square$

Remark 7 Selecting jobs from the longest chains maximizes the number of chains that remain available in subsequent periods.

The next result characterizes necessary and sufficient conditions needed for a feasible schedule to exist.

Lemma 4 A feasible solution to constraints (2) and (3) exists if and only if $\sum_{k \in \mathcal{T}} \bar{u}_k = n$.

($\Leftarrow$) Proof by induction. Suppose the period indices $i_1, \dots, i_T$ are sorted such that $s_{i_1} \leq \dots \leq s_{i_T}$.

I. From (5), period i_1 can assign $\bar{u}_1$ resources to chains $1, \dots, \bar{u}_1$.

II. Suppose periods $i_1, \dots, i_{k-1}$ assigns $\bar{u}_1, \dots, \bar{u}_{k-1}$ tasks on chains respectively, each selecting the chains in order with the most unassigned tasks at the time of assignment. From (5), $\bar{u}_k \leq \bar{q}_k$. As a result, there are enough chains with unscheduled tasks for period i_k to make $\bar{u}_k$ assignments. Further, $\bar{u}_k \leq L + a_{i_k}$. Thus, there exists a feasible solution to constraints (2) and (3) for periods $i_1, \dots, i_k$.

($\Rightarrow$) Lemma 3 and Remark 7 establish that if the chains with the most unassigned tasks are selected, then there is no restriction on the order in which the periods are chosen. In particular, they can be selected in the order $i_1, \dots, i_T$. If $\bar{u}_k = \bar{q}_k < a_{i_k} + L$, then $u_k + L - \bar{q}_k$ resources cannot be used by the chains and must be discarded. As a result, no more than $\sum_{k=1}^T \bar{u}_k \leq n$ resources can be assigned to the chains.

If $\sum_{k=1}^T \bar{u}_k < n$, then no feasible solution exists. $\square$

Lemma 5 Procedure LSubChainS finds an optimal schedule for Problem $PS|chains, p_j = 1, r_j = 1, a_t, s_t|R_{\max}$ with L resources, when one exists in $O(\max\{qT, q \log q, T \log T\})$ time.

Proof Given L resources, McNaughton’s “wrap-around” procedure finds a feasible assignment for any set of chains where $LT \geq n$ and $T \geq n_h$ for $h \in Q$. Consequently, the amount of subcontracting required is $\gamma = \max\{n - LT, 0\}$. Remark 5 shows that tasks can be scheduled by period. The periods are selected in nondecreasing subcontracting cost order. Lemma 3 shows that this order does not restrict the feasible set in any way. Since available resources are independent by period, Remark 6 shows that scheduling least cost subcontracting periods first is desirable because these periods have the most chains with tasks that require scheduling. Selecting periods in nonincreasing cost order and scheduling as much subcontracting as can be used, minimizes the total cost. This satisfies the budget constraint (4), when a feasible solution exists for L resources.

At a given iteration $k \in T$, $u = \min\{L + a_{i_k}, L + \gamma, q\}$ tasks are assigned resources and subcontracting. There are $L + a_{i_k}$ units of resource and contracting available. Also, exactly γ tasks require subcontracting. In accordance with Lemma 3, the u chains with the greatest number of unassigned tasks are selected for processing. Finally, at the current iteration k , at most q chains can be serviced. As a result, the value of u for period i_k is the value of $\bar{u}_k$ found in (5). Lemma 4 shows that this process finds a feasible solution to (P_{SC}) , when one exists.

To determine the computational effort, Step 1 requires $O(q \log q + T \log T)$ time. Step 2 is called $O(T)$ times and each application requires $O(q)$ computation. Step 3 requires $O(qT)$ time and is called only once. Hence, the overall computational time requirement of Algorithm LSubChainS is $O(\max\{qT, q \log q, T \log T\})$.

Theorem 5 An optimal schedule for problem $PS | chains, p_j = 1, r_j = 1, a_t, s_t | R_{\max}$ can be found in $O(\max\{qT, q \log q, T \log T\} \log q)$ time.

Proof Lemma 5 establishes that for a given L , the minimum cost schedule can be found in $O(\max\{qT, q \log q, T \log T\})$ time. Since there are q chains, $0 \leq L \leq q$. Solve Procedure LSubChainS using a binary search on $L \in \{1, 2, \dots, q\}$ to find the smallest L that has a total subcontracting cost no larger than B . This finds the optimal solution in $O(\max\{qT, q \log q, T \log T\} \log q)$ time. $\square$

However, the following result shows that even restricted versions of Subcontracting Problem B with inforest precedence constraints are intractable. The reduction is from the following problem which is known to be unary NP-complete (Garey and Johnson 1979).

3-Partition: Given a set S of elements with integer values $c_1, \dots, c_{3m}$, where $\sum_{i=1}^{3m} c_i = mb$ and $b/4 < c_i < b/2$ for $i = 1, \dots, 3m$, does there exist a partition of S into subsets $S_1, \dots, S_m$ such that $|S_i| = 3$ and $\sum_{j \in S_i} c_j = b$, for $i = 1, \dots, m$?

Theorem 6 The recognition version of problem $PS | inforest, p_j = 1, r_j = 1, s_t | R_{\max}$ is unary NP-complete.

Proof By reduction from 3-Partition (Garey and Johnson 1979) in pseudopolynomial time.

The proof uses some ideas from Garey et al. (1983). However, the lack of a machine profile for our problem requires different arguments. Let $K = 5m^2$. Consider an instance of problem $PS | inforest, p_j = 1, r_j = 1, s_t | R_{\max}$, where the planning horizon is

$$T = Kmb - (3m^2 - 5m)/2.$$

One machine is available to process tasks in periods $1, \dots, T$. Thus, the threshold value of the objective is $R_{\max} = 1$. The budget is

$$B = \sum_{i=1}^m (m - i + 1)[(Kb - 1) + 3(i - 1)].$$

Define periods

$$t_1 = 1,$$

$$t_{i+1} = t_i + Kb - 3(m - i) + 1, \quad i = 1, \dots, m - 1.$$

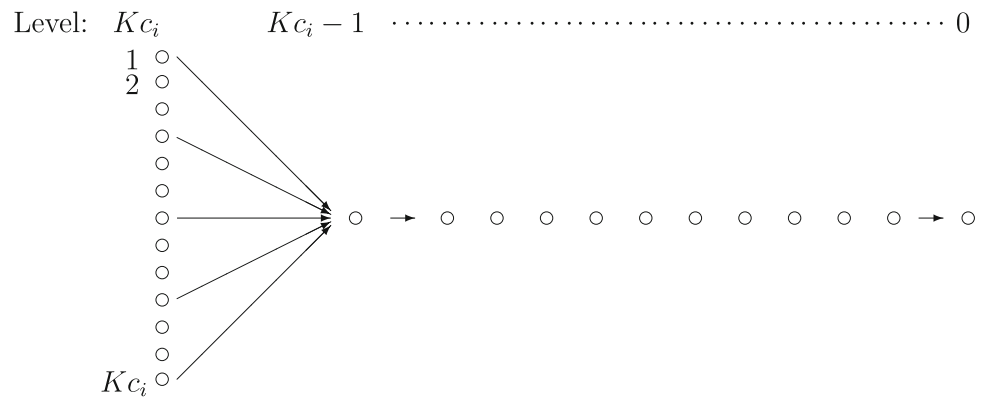
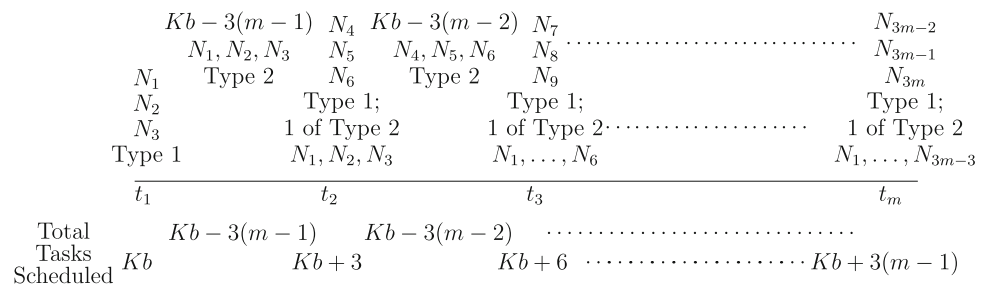
Let

$$s_t = \begin{cases} m - i + 1, & t = t_i \in V \equiv \{t_1, \dots, t_m\}, \\ B + 1, & t \in \bar{V} \equiv \{1, \dots, T\} \setminus V. \end{cases}$$

The precedence structure is an inforest consisting of $3m$ intrees, where intree N_i , for $i \in \{1, \dots, 3m\}$, contains Kc_i tasks of level Kc_i denoted as “Type 1_i ” tasks, and one task of each level $Kc_i - 1, Kc_i - 2, \dots, 0$, which are denoted as “Type 2_i ” tasks and form a chain. All of the Type 1_i tasks precede the level $Kc_i - 1$ Type 2_i task. Thus, $n = 2Kmb$. The structure for a typical intree N_i is shown in Fig. 1.

We show that there exists a feasible solution to the above instance of $PS | inforest, p_j = 1, r_j = 1, s_t | R_{\max}$ with subcontracting cost $\leq B$ if and only if there exists a solution to 3-Partition.

($\Rightarrow$) Assume that there exists a solution to 3-Partition. Let the subsets that form this solution contain the following elements: $S_1 = \{1, 2, 3\}, S_2 = \{4, 5, 6\}, \dots, S_m = \{3m - 2, 3m - 1, 3m\}$. Consider a schedule σ^0 constructed as follows. Schedule the $Kc_1 + Kc_2 + Kc_3 = Kb$ Type $1_1, 1_2$ and 1_3 tasks in period t_1 . This incurs cost $m(Kb - 1)$, since one task is scheduled on the machine and the other $(Kb - 1)$ tasks are subcontracted. Schedule $Kc_i - (m - 1)$ Type 2_i tasks, for $i = 1, 2, 3$, in the time periods $t_1 + 1, \dots, t_2 - 1$, one in each time period on the machine in their precedence order. Schedule the remaining $3(m - 1)$ Type 2_i tasks, one of each type i in periods $t_2, \dots, t_m$, for $i = 1, 2, 3$ in their precedence order. Next, schedule each set of three Type 1_i tasks in the intrees $N_{3i-2}, N_{3i-1}, N_{3i}$, for $i = 2, \dots, m$ in period t_i . Also, for $i = 2, \dots, m - 1$, schedule the first $Kc_i - (m - i)$ Type 2_i tasks in periods $t_i + 1, \dots, t_{i+1} - 1$, and schedule the remaining Type $2_{3i-2}, 2_{3i-1}, 2_{3i}$ tasks one in each period $t_{i+1}, \dots, t_m$. Finally, schedule the Type $2_{3m-2}, 2_{3m-1}, 2_{3m}$

Fig. 1 Intree N_i **Fig. 2** Schedule Associated with a Solution of 3-Partition

tasks one in each period $t_m + 1, \dots, T$. This schedule is shown in Fig. 2, where we use the format “ N_i Type 1” to represent *all* the Type 1 tasks of intree N_i being processed concurrently, and the format “ z N_i Type 2” to represent a subset of cardinality z of the Type 2 tasks of intree N_i being processed sequentially in their precedence order. As mentioned above, between times t_i and t_{i+1} , for $1 \leq i \leq m - 1$, only one task is scheduled in each period.

Because the precedence order of each task is satisfied, the schedule is feasible. Further, intervals $t_i + 1, \dots, t_i + Kb - 3(m - i)$, for $i = 1, \dots, m$, have only one task scheduled. Hence, no subcontracting is required and no cost is incurred at those times. However, at times t_i , for $i = 1, \dots, m$, there are $Kb + 3(i - 1)$ tasks scheduled, and therefore $(Kb - 1) + 3(i - 1)$ tasks are subcontracted. As a result, schedule σ^0 incurs a subcontracting cost of

$$m[Kb - 1] + (m - 1)[(Kb - 1) + 3] + \dots + 1[(Kb - 1) + 3(m - 1)] = B.$$

($\Leftarrow$) Assume that there exists a feasible schedule σ to the above instance of problem $PS \mid intree, p_j = 1, r_j = 1, s_t \mid R_{\max}$ where y_t denotes the number of tasks subcontracted in period t , for $t = 1, \dots, T$, and $\sum_{t=1}^T s_t y_t \leq B$. Because there is only one machine and $s_t = B + 1$ for $t \in \bar{V}$, schedule σ processes at most one task in these periods. Consequently, all subcontracting must occur in periods $t_1, \dots, t_m$.

Suppose that less than Kb Type 1 tasks are processed in period t_1 . The inforest structure of the precedence constraints

implies that there are at least $Kb(m - 1) + K$ Type 2 tasks unprocessed by the start of period t_2 . Since not more than one Type 2 task of the same intree can be processed in the same period, no more than $3m(m - 1)$ of these tasks can be processed in the $(m - 1)$ periods $t_2, \dots, t_m$. Further, there are only $Kb(m - 1) - 3(m - 1)(m - 2)/2$ periods of $\bar{V}$ after period t_2 that can process at most one task in a feasible schedule. Since

$$Kb(m - 1) + K > 3m(m - 1) + Kb(m - 1) - 3(m - 1)(m - 2)/2,$$

no feasible solution is possible. This implies that at least Kb Type 1 tasks must be processed in period t_1 .

By a similar analysis, it can be shown that at least Kbi tasks must be processed in periods $t_1, \dots, t_i \in V$. Below we use the fact that, if *exactly* Kb Type 1 tasks are processed in $t_i \in V$, their cost is

$$\sum_{i=1}^m (m - i + 1)(Kb - 1) = m(m + 1)(Kb - 1)/2. \quad (6)$$

We now show that an optimal schedule processes no more than Kb Type 1 tasks in period t_1 . First, we consider only Type 1 tasks that are subcontracted. Since $s_i = B + 1$ for $t_i \in \bar{V}$, this subcontracting occurs only in periods $t_i \in V$. Suppose that for some $i \in \{1, \dots, 3m\}$, the subcontracted Type 1 _{i} tasks are scheduled in multiple periods of V . Since none of the Type 2 _{i} tasks can be processed until all of the Type 1 _{i} tasks are completed, all of the Type 1 _{i} tasks can

be feasibly processed in the last of these multiple periods of V without any change to the remainder of the schedule. Moreover, the strictly decreasing subcontracting costs imply that subcontracting all of Type 1_{*i*} tasks in the last of these multiple periods reduces subcontracting costs. As a result, we assume that for $i \in \{1, \dots, 3m\}$, the subcontracting set of Type 1_{*i*} tasks occurs in one period.

Let $J \subseteq \{1, \dots, 3m\}$ denote the set of indices of intrees where some Type 1 tasks are processed in t_1 . If more than Kb Type 1 tasks are processed in this period in an optimal schedule, then $\sum_{i \in J} Kc_i \geq Kb + K$. If all of these Type 1 tasks are processed in period t_1 , then the cost of subcontracting grows by at least $(s_1 - \max_{i \geq 2} \{s_i\})K = (s_1 - s_2)K = K$. From (6), without using the periods of $\bar{V}$, the subcontracting cost for the Type 1 tasks alone is $m(m+1)(Kb-1)/2 + K > B$. Hence, schedule σ is infeasible. However, it may be possible to process some of the Type 1 tasks of $\cup_{i \in J} N_i$ in the periods of $\bar{V}$ to reduce the subcontracting cost. We now show that the cost reduction from using the intervals of $\bar{V}$ is insufficient to satisfy the budget constraint.

Since there are $3m$ Type 2 chains and $|V \setminus \{t_1\}| = m - 1$, it is not possible to subcontract more than $3m(m-1)$ Type 2 tasks in the periods of $V \setminus \{t_1\}$. Since $|\bar{V}| = T - |V| = [Kmb - (3m^2 - 5m)/2] - m = Kmb - 3(m^2 - m)/2$ and there are Kmb Type 2 tasks, the number of available periods of $\bar{V}$ to process Type 1 tasks is no larger than $Kmb - 3(m^2 - m)/2 - (Kmb - 3m(m-1)) = 3(m^2 - m)/2$. Consequently, at least $Kb + K - 3(m^2 - 5m)/2$ of the Type 1 tasks of $\cup_{i \in J} N_i$ tasks must be processed in period t_1 , with a cost of $m[Kb + K - 3(m^2 - m)/2]$. The reduction in subcontracting cost is not more than

$$3m(m-1)[(m-1)-1],$$

where the last term compares the cost of subcontracting in periods t_2 and t_m . As a result, the additional cost to process more than Kb tasks in period t_1 is at least

$$m[K - 3(m^2 - m)/2] - 3m(m-1)(m-2) > 0.$$

Hence, it is not optimal to process more than Kb tasks in this period.

A similar analysis can be presented to show that periods $t_2, \dots, t_m$ cannot process more than Kb Type 1 tasks.

Finally, consider the intrees that provide exactly Kb Type 1 tasks at period t_i , for $i = 1, \dots, m$. It is shown above that these are the full sets of Type 1 tasks for a subset of the $3m$ given intrees. Then, since by assumption $b/4 < c_i < b/2$, $i = 1, \dots, 3m$, these represent all of the Type 1 tasks of exactly three intrees. It follows that the indices of these intrees form a solution to 3-Partition. $\square$

5 Concluding Remarks

This paper considers the scheduling of limited resources in the context of project management problems with a constraint on the project makespan. A practical problem that is central to our work is the scheduling of a project to minimize the maximum resource load over the life of the project. We consider several variants of this problem, for which we either describe an efficient optimal algorithm or provide a proof of intractability. We also consider a variation of the resource loading problem where work can be subcontracted at cost. This cost can be defined either by the job being subcontracted or by the time at which the subcontracting takes place. Overall, our results provide a detailed mapping of the solvability of resource-constrained scheduling problems with a variety of practical characteristics.

Our results show that the solvability of maximum resource load minimization problems is particularly sensitive to several details of the environment. These include whether task times are unit or arbitrary, whether resource requirements are unit or binary, whether the project network is a chain, an opposing forest, or a tree, whether subcontracting costs for jobs are unit or binary, and whether the amount of subcontracting at a time period is constrained.

Several interesting problems remain open for future research. First, the problem with unit processing times and resource requirements, but binary time-based subcontracting costs, remains open. Second, the complexity of related problems with series-parallel precedence constraints can be studied. Third, the design of effective heuristics for the problems we identify as intractable is beyond the scope of this work, but valuable to pursue. Fourth, for related problems with arbitrary task times, we recommend studying the effect of allowing a limited number of task preemptions, as motivated by operational constraints. Finally, the study of project management problems with constraints other than makespan should provide additional perspectives on this theoretically and practically important problem class.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Blazewicz, J., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1983). Scheduling subject to resource constraints: Classification and complexity. *Discrete Applied Mathematics*, 5, 11–24.
- Browning, T. R., & Yassine, A. A. (2010). Resource-constrained multi-project scheduling: Priority rule performance revisited. *International Journal of Production Economics*, 126, 212–228.
- Burgess, A. R., & Killebrew, J. B. (1962). Variation in activity level on a cyclical arrow diagram. *Journal of Industrial Engineering*, 13, 76–83.
- Crail, C. (2024). Payroll outsourcing in 2024: The ultimate guide. Forbes.com. August 17. Available at: <https://www.forbes.com/advisor/business/payroll-outsourcing/>. Last accessed: August 21, 2024.
- Demeulemeester, E. (1995). Minimizing resource availability costs in time-limited project networks. *Management Science*, 41, 1590–1598.
- Demeulemeester, E. L., & Herroelen, W. S. (2002). *Project scheduling: A research handbook*. Boston, MA: Kluwer.
- Drexl, A., & Kimms, A. (2001). Optimization guided lower and upper bounds for the resource investment problem. *Journal of the Operational Research Society*, 52(3), 340–351.
- Du, J., Leung, J.Y.-T., & Young, G. H. (1991). Scheduling chain-structured tasks to minimize makespan and mean flow time. *Information and Computation*, 92, 219–236.
- Elwany, M. H., Korish, I. E., Barakat, M. A., & Hafez, S. M. (1998). Resource smoothing in repetitive projects. *Computers and Industrial Engineering*, 35, 415–418.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. New York: Freeman.
- Garey, M. R., Johnson, D. S., Tarjan, R. E., & Yannakakis, M. (1983). Scheduling opposing forests. *SIAM Journal on Algebraic Discrete Methods*, 4, 72–93.
- Graham, R., Lawler, E. L., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1979). Optimization and approximation in deterministic machine scheduling: A survey. *Annals of Discrete Mathematics*, 5, 287–326.
- Grötschel, M., Lovász, L., & Schrijver, A. (1988). *Complexity, Oracles, and Numerical Computation*. Geometric algorithms and combinatorial optimization: Springer. 0-387-13624-X
- Hobbs, B., & Menard, P. (1993). Organizational choices for project management. In *The AMA Handbook of Project Management*, ed. P.C. Dinsmore, AMACON, New York.
- Hu, T. C. (1961). Parallel sequencing and assembly line problems. *Operations Research*, 9, 841–848.
- Hu, X., Cui, N., & Demeulemeester, E. (2015). Effective expediting to improve project due date and cost performance through buffer management. *International Journal of Production Research*, 53(5), 1460–1471.
- Kartam, N., & Tongthong, T. (1998). An artificial neural network for resource leveling problem. *Artificial Intelligence for Engineering Design, Analysis & Manufacturing: AIEDAM*, 12, 273–287.
- Kolisch, R. (1999). Resource allocation capabilities of commercial project management software packages. *Interfaces*, 29, 19–31.
- Kyriklidis, C., & Dounias, G. (2016). Evolutionary computation for resource leveling optimization in project management. *Integrated Computer-Aided Engineering*, 23, 173–184.
- McNaughton, R. (1959). Scheduling with deadlines and loss functions. *Management Science*, 6, 1–11.
- Möhring, R. H. (1984). Minimizing costs of resource requirements in project networks subject to a fixed completion time. *Operations Research*, 32(1), 89–120.
- Muntz, R. R., & Coffman, E. G., Jr. (1970). Preemptive scheduling of real-time tasks on multiprocessor systems. *Journal of the ACM*, 17, 324–338.
- Neumann, K., & Zimmermann, J. (1999). Resource levelling for projects with schedule-dependent time windows. *European Journal of Operational Research*, 117, 591–605.
- Pisano, G. P., & Verganti, R. (2008). Which kind of collaboration is right for you? *Harvard Business Review*, 86, 78–86.
- Prahalad, C. K., & Hamel, G. (1990). The core competence of the corporation. *Harvard Business Review*, 68, 79–91.
- Project Management Institute. (2013). *A Guide to the Project Management Body of Knowledge (PMI Guide)*. Newton Square, PA: PMI Publications.
- Quinn, J. B., & Hilmer, F. G. (1994). Strategic outsourcing. *Sloan Management Review*, 35, 43–55.
- Raja, K., & Kumanan, S. (2007). Resource leveling using Petrinet and memetic approach. *American Journal of Applied Sciences*, 4, 317–322.
- Rieck, J., & Zimmermann, J. (2014). Exact methods for resource leveling problems. in *Handbook on Project Management and Scheduling*, 361–387. Springer, Switzerland.
- Sturgeon, T. J. (2002). Modular production networks: A new American model of industrial organization. *Industrial and Corporate Change*, 11, 451–496.
- Tully, S. (1994). You'll never guess who really makes . . . *Fortune*, 130, 124–129.
- Younis, M. A., & Saad, B. (1996). Optimal resource leveling of multi-resource projects. *Computers and Industrial Engineering*, 31, 1–4.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.